



Sistema de Gerenciamento de Requisitos

Nexus – Engenharia de Software

Eduardo Lemes – Guilherme Batista – Lucas Marcelino – Ruan Almagro



Introdução

O cenário atual do desenvolvimento de software exige das empresas rapidez, organização e eficiência no gerenciamento de requisitos. Grandes companhias que lidam com múltiplos clientes enfrentam o desafio de documentar, versionar e reaproveitar requisitos de maneira clara e estruturada.



A Empresa



Médio porte, softwares sob medida, qualidade e rastreabilidade.



Especializada em Gestão Empresarial, Sistemas PDVs e ERPs



Parceiro do Instituto Federal de São Paulo, auxilia com estagios e patrocínio na SEMAC



Inovação contínua, metodologias ágeis, entrega de soluções confiáveis e evolutivas



Problemática

A Algomith Software atende diversos clientes com projetos simultâneos e personalizados. Tais projetos muitas vezes compartilham requisitos semelhantes, isso gera a necessidade de reuso ou versionamento. Sem uma ferramenta específica, que trate os requisitos com rastreabilidade, versionamento e reuso, esses requisitos acabam se tornando duplicados, inconsistentes ou mal documentados, prejudicando a produtividade e a qualidade dos sistemas entregues.



Sistema de Gestão & Versionamento de Requisitos

Criado para
transformar a
Algomith

Semelhante ao GitHub
Commits e Branches

Histórico de autor,
data e justificativa.

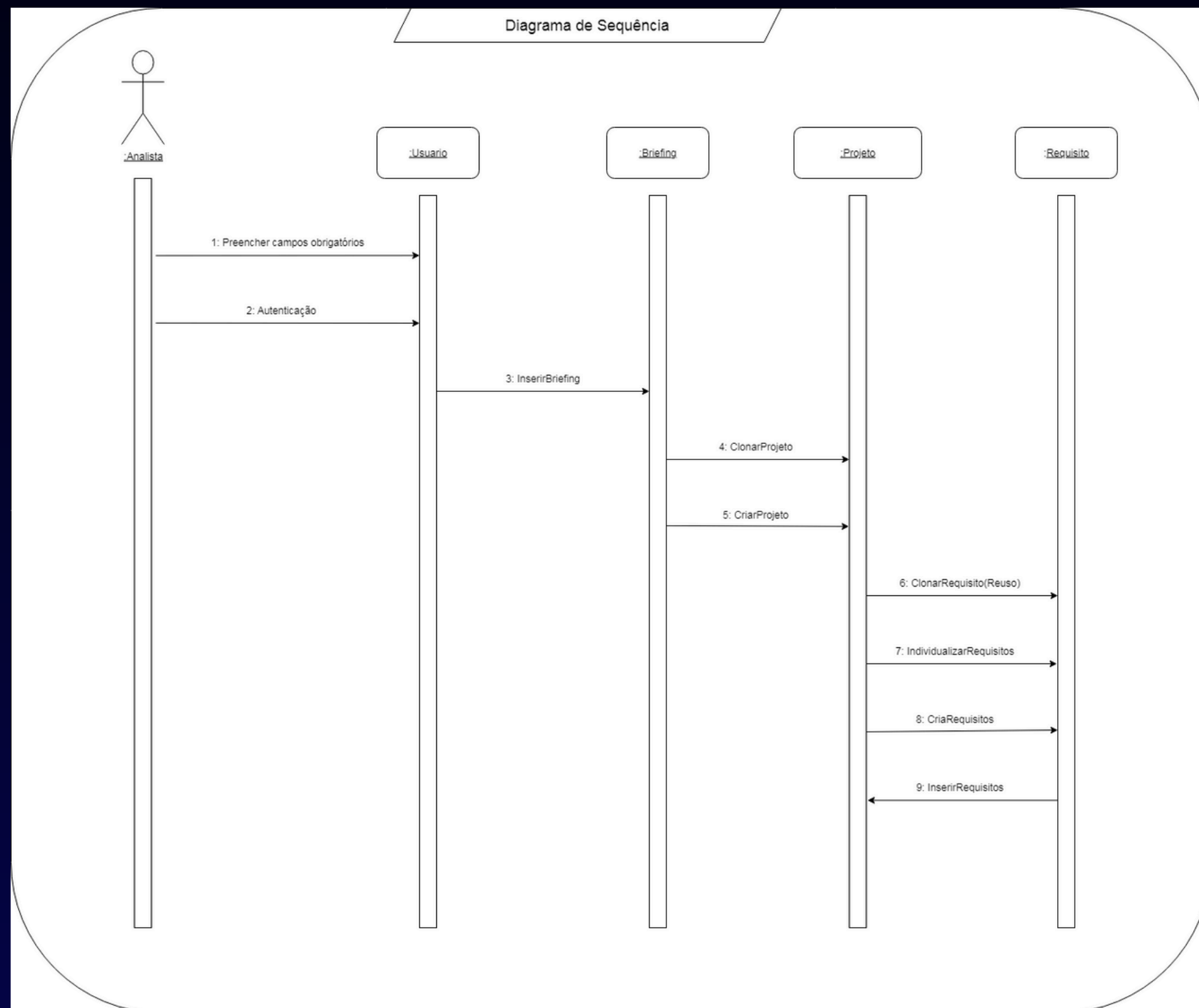
DIFERENCIAL

IA Integrada
sugerindo requisitos,
identificando duplicatas,
propondo rascunhos

um repositório vivo de conhecimento, com rastreabilidade completa, reuso inteligente e evolução contínua – trazendo para os requisitos a mesma segurança e eficiência do versionamento de código.



Fluxo Principal





Modelo de Processo Desenvolvimento

Para a construção do Sistema de Gestão e Versionamento de Requisitos (SGR), adotaremos um Modelo de Processo Evolucionário, Iterativo e Incremental, em razão da necessidade de entregas frequentes, funcionais e que possam ser avaliadas em tempo real.

O aspecto incremental garante que cada ciclo entregue um pedaço do sistema que seja funcional e utilizável, ainda que não definitivo. Esses incrementos são projetados para se integrar ao todo, formando gradualmente um sistema robusto. Além disso, o processo prevê que feedback contínuo seja parte essencial da construção.



Metodologia Ágil

A Nexus Engenharia de TI adotou a Metodologia Ágil Orbit (MAO), um framework original criado pela própria equipe para o desenvolvimento do SGR. Ela foi concebida para atender às necessidades de entregas contínuas e também renovação constante do que já foi desenvolvido. O trabalho é organizado em “órbitas de valor”, ciclos curtos e planejados com objetivos definidos pelo cliente. Em cada órbita, o sistema evolui com novas funcionalidades e revisa entregas anteriores, garantindo consistência e qualidade.

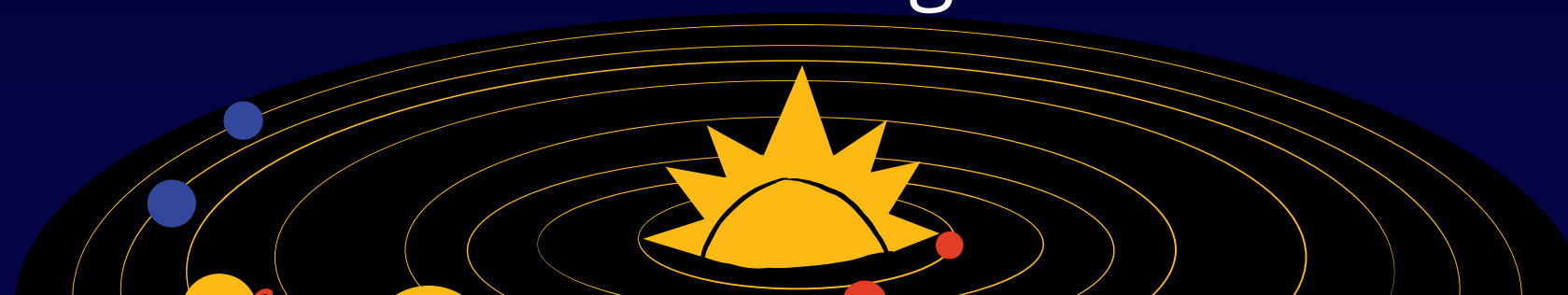




Orbit

A Orbit estrutura-se em quatro funções complementares e interdependentes, responsáveis por manter a estabilidade do fluxo orbital.

- O Sun é o cliente — no caso, a Algomith — e representa o centro de gravidade do processo. Todas as decisões e prioridades partem dele, sendo o eixo em torno do qual as atividades da equipe gravitam.
- Os Satellites são os membros da equipe que trabalham diretamente no desenvolvimento
- O Orbit Master é o gerenciador da metodologia, garantindo que o fluxo orbital se mantenha estável, ou seja, responsável por alinhar a equipe com os princípios e objetivos definidos.
- O Gravity Mediator tem o papel essencial de capturar, organizar e traduzir o feedback do cliente em backlog acionável.

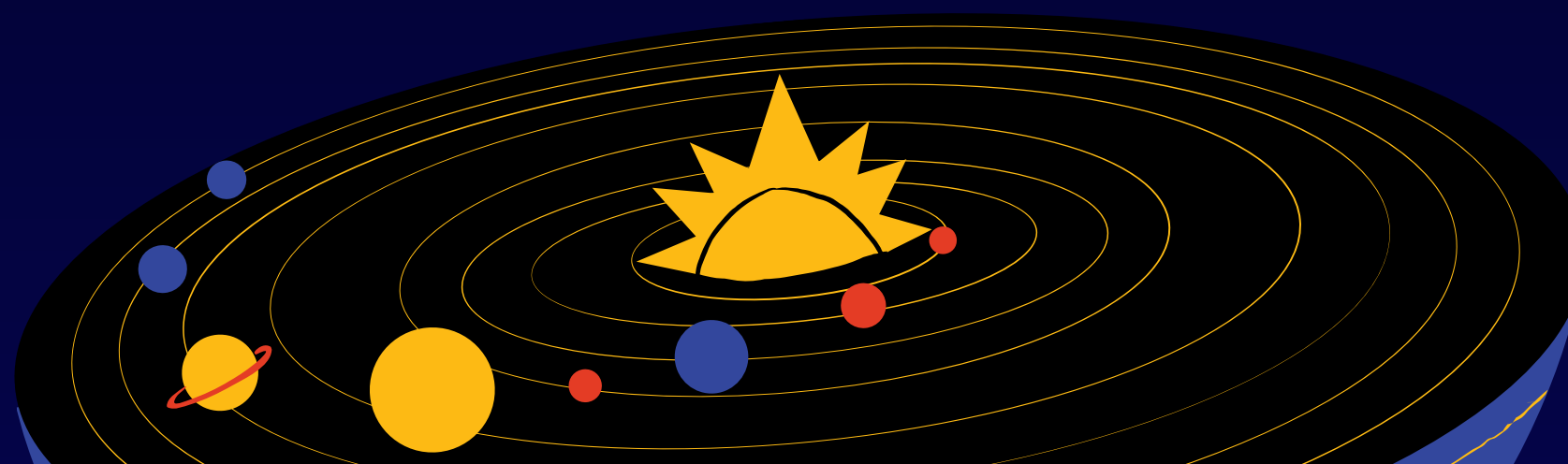




Orbit

Princípios

- O primeiro é a Ciclicidade de valor real, que garante que, a cada órbita de duas semanas, a Algomith receba entregas revisadas e renovadas.
- O segundo princípio é a Colaboração com o cliente, que garante que os objetivos de negócio sejam constantemente revisitados e validados.
- O terceiro princípio é a Renovação contínua, onde parte do esforço, em que cada órbita, é direcionado ao refinamento de entregas anteriores.
- O quarto princípio é a Medição de progresso baseada em software funcional e validado.



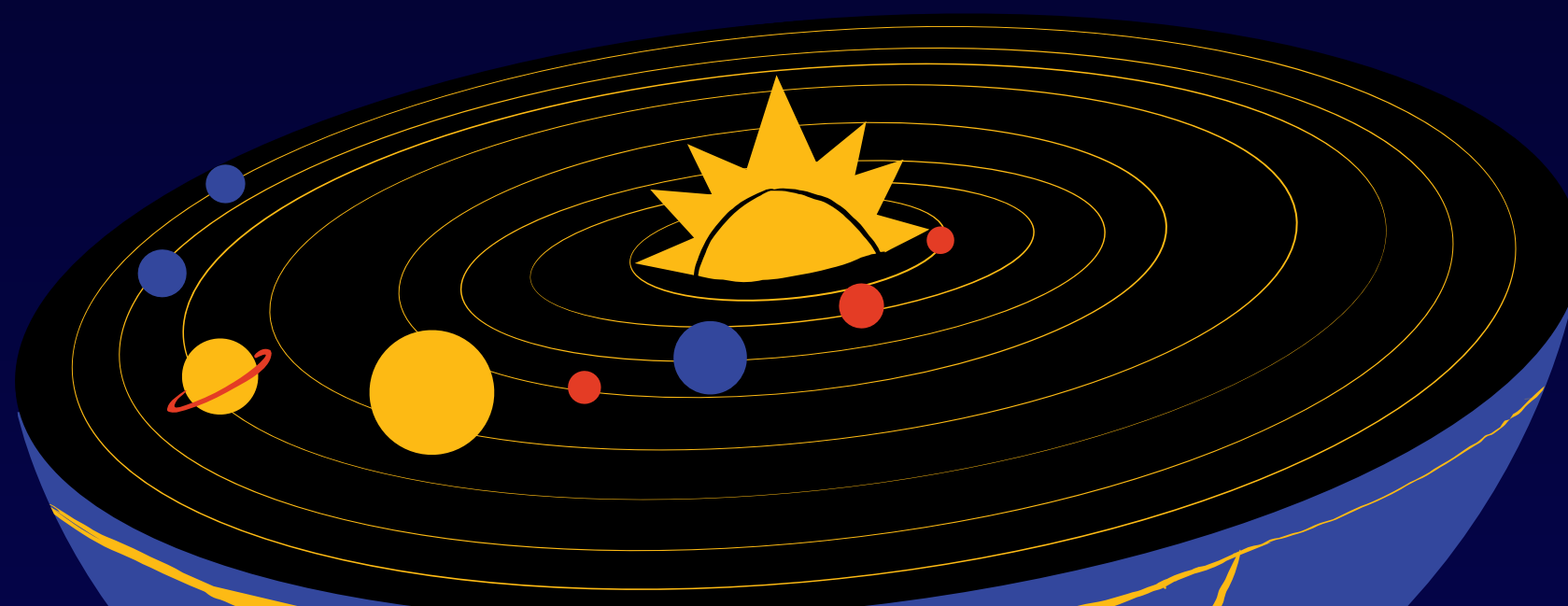


Orbit

Estrutura de trabalho

A estrutura da MAO é organizada em ciclos de órbitas quinzenais, cada um dividido em três fases:

- Alignment: definição clara dos objetivos da órbita e priorização de entregas.
- Execution: implementação de novas funcionalidades acompanhada de revisões em entregas passadas.
- Return to the Sun: validação final junto ao cliente, assegurando que tudo conforme palenjado.





Orbit

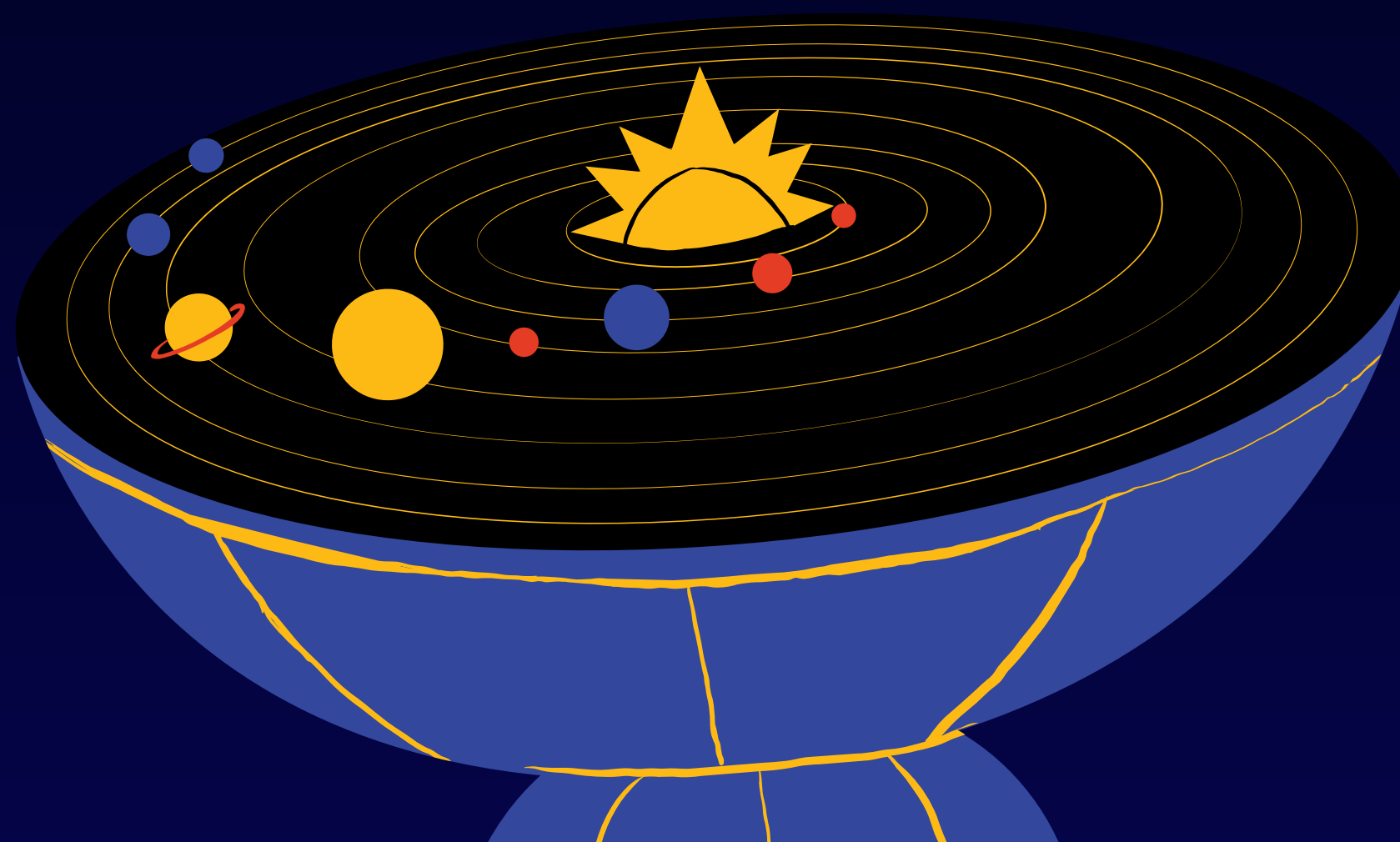
Partes Entregáveis

1 Módulo de autenticação e cadastro de projetos, criando a base estrutural.

2 Módulo de briefing e indexação de conteúdo pela IA

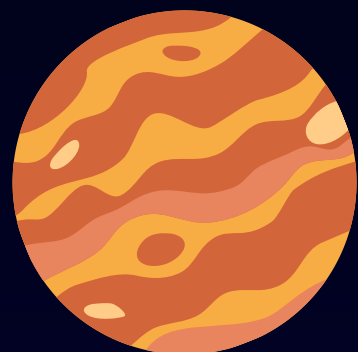
3 Módulo de requisitos, com IA atuando como motor de sugestões

4 Governança de mudanças e exportação de requisitos





Qualidade do Software



Confiabilidade

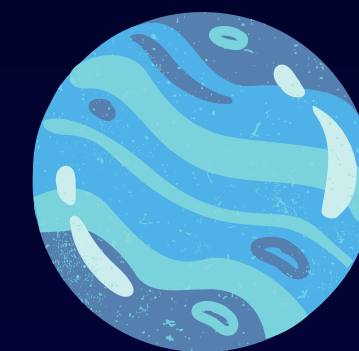
O sistema precisa estar sempre disponível e consistente, sem perda de dados ou falhas em commits. Ao salvar um novo requisito, por exemplo, o sistema gera automaticamente um hash único e mantém o histórico de versões, portanto, mesmo que ocorra uma falha de rede, o registro anterior permanece íntegro no servidor.

Segurança

Garante a proteção dos dados e o acesso apenas por usuários autorizados por meio uso de autenticação multifator, criptografia de senhas e níveis de permissão (gestor, analista, desenvolvedor). Um analista, por exemplo, não consegue excluir requisitos sem autorização do gestor.

Eficiência

Mede o desempenho e o tempo de resposta do sistema em diferentes cenários. Assim, mesmo com milhares de requisitos armazenados, a IA de similaridade deve sugerir resultados relevantes em segundos, otimizando o tempo do analista durante a elicitación.





Qualidade do Software



Usabilidade

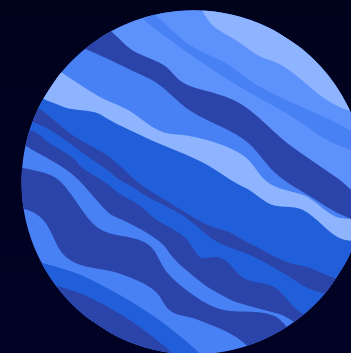
Relaciona-se à facilidade de uso e à experiência do usuário. Exemplo: o sistema tem interface intuitiva e responsiva, com menus claros, atalhos de ação rápida e layout adaptável a telas menores, reduzindo a curva de aprendizado para novos usuários.

Manutenabilidade

Indica a facilidade de realizar correções ou atualizações sem afetar outras partes do sistema. Por exemplo: cada requisito versionado está isolado em um branch independente, permitindo corrigir apenas aquele item sem impactar o restante do projeto. Além disso, o projeto é documentado automaticamente pelo Sphinx, que, juntamente com a IA, facilita a manutenção.

Portabilidade

Refere-se à capacidade do sistema de funcionar em diferentes ambientes e dispositivos. O projeto é uma aplicação web responsiva, acessível via desktop, notebook ou tablet, em navegadores como Chrome, Edge ou Firefox, sem necessidade de instalação local.



Documentação

A documentação do SGR será conduzida de forma integrada ao desenvolvimento, adotando boas práticas de clareza e padronização. Todo o código-fonte será comentado de forma objetiva, seguindo convenções para explicar a lógica de funções e classes. Essa prática é essencial para garantir que novos desenvolvedores compreendam as implementações e facilita a manutenção do sistema a longo prazo. A documentação não serve apenas à equipe, mas também será utilizada pela Algomith para auditoria, rastreabilidade e transferência de conhecimento.



Código Comentado

Facilita a compreensão e manutenção a longo prazo.



Auditoria e Rastreabilidade

Essencial para a Algomith e transferência de conhecimento.

Documentação

O projeto utilizará o Sphinx Documentation como ferramenta principal para gerar automaticamente manuais de referência técnica. Através da anotação de docstrings em módulos, o Sphinx produzirá documentação em JSON/HTML. Assim que gerados, o SGR receberá os arquivos. A Inteligência Artificial (IA) do sistema realizará a leitura semântica desse conteúdo, extraindo descrições de classes e funções. Esses dados serão vinculados aos requisitos versionados, garantindo que as implementações técnicas estejam sempre alinhadas às especificações registradas.



Geração Automática

Sphinx cria documentação técnica.



Leitura Semântica

IA extrai descrições de classes e funções.



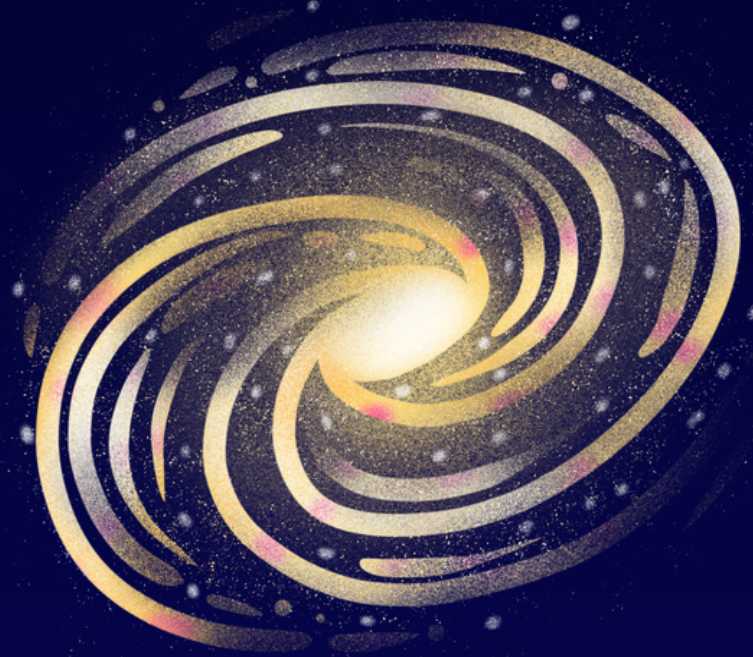
Vínculo com Requisitos

Alinhamento com especificações versionadas.



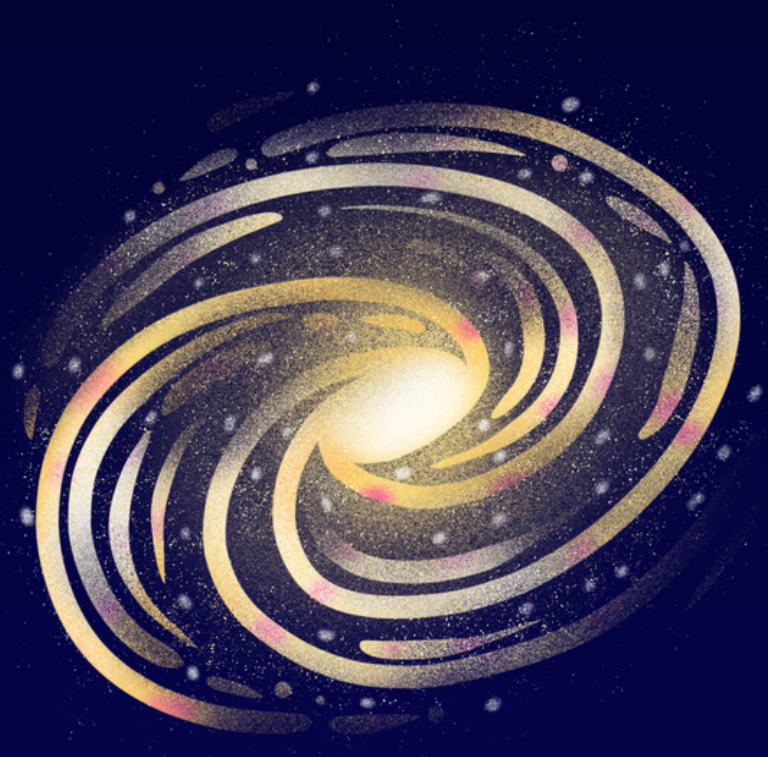
Estratégia de Teste

A estratégia de teste assegura a qualidade e confiabilidade do SGR, sendo fundamentada nos princípios de Verificação e Validação (V&V) descritos por Pressman. O processo é contínuo, incremental e iterativo, integrando-se diretamente à Metodologia Ágil Orbit (MAO). A equipe adapta o modelo em "V" para a estrutura cíclica da MAO, garantindo que cada fase do desenvolvimento tenha um nível de teste correspondente. Os quatro níveis são: Testes Unitários (validação individual de componentes), Testes de Integração (verificam a comunicação entre módulos), Testes de Sistema (simulam o comportamento completo do usuário, cobrindo o fluxo principal) e Testes de Aceitação (validação final com a Algomith).





Estratégia de Teste



Utilizamos uma combinação de abordagens: Testes de Caixa Branca são aplicados pelos desenvolvedores para verificar a lógica interna, enquanto os Testes de Caixa Preta avaliam o comportamento externo do sistema e a conformidade com requisitos. A integração dos módulos segue uma estratégia Sanduíche (Híbrida), que combina Top-Down e Bottom-Up para permitir o desenvolvimento paralelo. A prática de "Shift-Left Testing" antecipa os testes para as fases iniciais, reduzindo custos e aumentando a eficiência. A qualidade é automatizada através de uma infraestrutura de Integração Contínua (CI), com GitHub Actions/Jenkins executando validações a cada commit.

Testes de Caixa Branca

Desenvolvedores verificam a lógica interna.

Testes de Caixa Preta

Avaliam comportamento externo e conformidade.

Estratégia Sanduíche

Combina Top-Down e Bottom-Up para desenvolvimento paralelo.

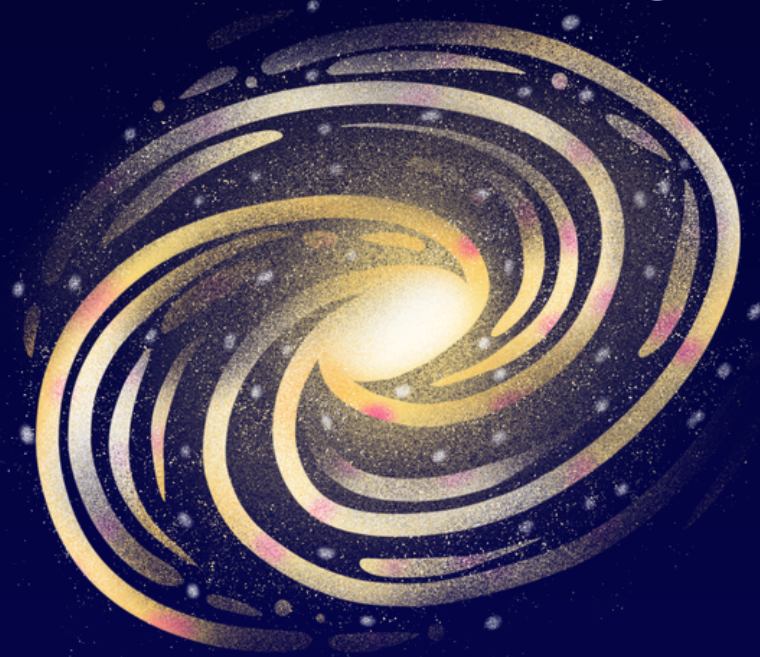
"Shift-Left Testing"

Antecipa testes para fases iniciais, reduzindo custos.



Estratégia de Teste

A Inteligência Artificial, componente central do SGR, possui uma estratégia de testes específica focada em seu desempenho semântico. Aplicamos métricas como o F1-Score para avaliar a precisão das sugestões de requisitos e a eficácia na detecção de duplicidades. A consistência das respostas da IA é verificada com testes automatizados de coerência linguística e contexto. A execução dos testes é responsabilidade da célula de Software Quality Assurance (SQA) da Nexus, que se integra à MAO. O Orbit Master supervisiona todo o processo de qualidade, e o Gravity Mediator coleta o feedback da Algomith e o traduz em casos de teste acionáveis. As métricas de controle (ex.: taxa de regressão, densidade de defeitos) são revisadas com o cliente na fase Return to the Sun, garantindo a entrega de valor contínuo.



Muito
Obrigado